and logic circuits. We saw that all Boolean functions/expressions had certain normal representations, such as minterm expansions, which means that Boolean logic circuits can be constructed using only the most basic logic gates AND, OR, and NOT. Theoretically, this means that the equational part of *Boolean Algebra* can be decided by truth tables, showing important connections between *Boolean Algebra*, *Propositional Logic*, and *Set Theory*.

Besides using *Boolean Algebra* to symbolize logic circuits, Shannon showed that we can construct logic circuits to do ordinary computations, a significant development underlying twentieth-century computer calculations.

Finally, we looked at some systematic ways to simplify Boolean expressions. Karnaugh maps provide a visual method for handling simple functions, while the Quine-McCluskey method gives a powerful tabular technique that can be developed into an efficient algorithm implemented on a computer. Simplification of Boolean expressions is obviously important to anyone designing complex logic circuits, both in engineering generally and computer science in particular.

Thus, a topic (*Boolean Algebra*) that is fairly abstract and theoretical is also very concrete and practical, forming the theoretical basis of some of the most important technological developments of the last century. This field demonstrates the interplay between these different aspects of mathematics, attracting people with varied interests and attitudes toward theoretical and practical mathematics. Given the concrete applications it has, *Boolean Algebra* makes an excellent first introduction to advanced abstract mathematics. And, given its links to *Propositional Logic* and *Set Theory*, it also helps unify different aspects of discrete mathematics in a foundational manner, providing reinforcement and a theoretical context for those fields.

EXERCISE SET 7.6

Exercises 1–6: Simplifying Boolean Functions via Boolean Algebra
Simplify the following functions using Boolean Algebra. Explain, by comparing the number of literals and number of operations involved, why your final answer is simpler than the given formula.

7.6.1. $f(x, y) = x\overline{y} + \overline{x}\,\overline{y}$

7.6.2. $f(x, y) = xy + x\overline{y} + \overline{x}\,\overline{y}$

7.6.3. $f(x, y, z) = xy + x\overline{y}\,\overline{z} + \overline{x}yz$

7.6.4. $f(x, y, z) = xy + x\overline{y}z + x\overline{z} + \overline{x}\,\overline{y}z + yz$

7.6.5. $f(x, y, z) = xy + xyz + xy\overline{z} + \overline{x}yz + \overline{x}y\overline{z}$

7.6.6. $f(w, x, y, z) = wz + \overline{w}y\overline{z} + x\overline{y}\,\overline{z} + \overline{x}\,\overline{y}\,\overline{z}$

Exercises 7–10: True or False

Are the following statements true or false? Explain your answer.

7.6.7. The minterm expansion for a Boolean function yields a simplified expression for the function.

7.6.8. An implicant for a Boolean function is one of the function's minterms.

7.6.9. An essential prime implicant for a Boolean function is a term that must appear in a simplified sum-of-products representation of the function.

7.6.10. The only terms that appear in a simplified sum-of-products representation of a Boolean function are essential prime implicants.

Exercises 11–14: Determining Implicants for K-Maps

For the following K-maps, identify the following, using alphabetic order $\{x, y\}$, $\{x, y, z\}$, *and* $\{w, x, y, z\}$ *for two, three, and four variables*:

a. *all the implicants of the associated Boolean function*;
b. *all the function's prime implicants*; *and*
c. *all the function's essential prime implicants.*

7.6.11.

	0	1
0	1	1
1	1	

7.6.12.

	00	01	11	10
0	1		1	1
1	1			1

7.6.13.

	00	01	11	10
0	1			1
1	1	1	1	1

7.6.14.

	00	01	11	10
00	1	1	1	1
01		1		
11		1	1	
10		1	1	1

Exercises 15–18: Simplifying Boolean Functions from K-maps
For each of the following K-maps, determine:
a. *the minterm expansion for the associated Boolean function, and*
b. *a simplified sum-of-products-of-literals representation for the minterm expansion.*

7.6.15. The K-map of Exercise 11
7.6.16. The K-map of Exercise 12
7.6.17. The K-map of Exercise 13
7.6.18. The K-map of Exercise 14

Exercises 19–28: Simplifying Boolean Functions Using K-maps
Draw K-maps for the following Boolean functions. Then use these K-maps to determine simplified sums of products for the given functions.

7.6.19. $f(x, y) = xy + \overline{x}$
7.6.20. $f(x, y) = xy + x\overline{y} + \overline{x}\,\overline{y}$
7.6.21. $f(x, y, z) = xyz + xy\overline{z} + \overline{x}\,\overline{y}z + \overline{x}\,\overline{y}z$
7.6.22. $f(x, y, z) = xy + x(\overline{y + z}) + \overline{x}\,\overline{y}z + \overline{y}\,\overline{z}$
7.6.23. $f(x, y, z) = xyz + \overline{x}\,\overline{y}z + \overline{x}\,\overline{z} + yz + \overline{y}\,\overline{z}$
7.6.24. $f(x, y, z) = x\overline{y}\,\overline{z} + \overline{x}y\overline{z} + \overline{x}\,\overline{y}z + \overline{x}\,\overline{y}\,\overline{z}$
7.6.25. $f(x, y, z) = xyz + xy\overline{z} + x\overline{y}z + x\overline{y}z + \overline{x}\,\overline{y}z + \overline{x}\,\overline{y}\,\overline{z}$
7.6.26. $f(w, x, y, z) = wxz + w\overline{x}\,\overline{y}\,\overline{z} + xz + y\overline{z}$
7.6.27. $f(w, x, y, z) = w\overline{x}y\overline{z} + w\overline{x}\,\overline{y}\,\overline{z} + \overline{w}xyz + \overline{w}\,\overline{x}y\overline{z} + \overline{w}\,\overline{x}\,\overline{y}\,\overline{z}$
7.6.28. $f(w, x, y, z) = wxy + x\overline{y}z + wy\overline{z} + wxyz + \overline{w}x\overline{y}\,\overline{z}$

Exercises 29–36: Quine-McCluskey Method
Use the Quine-McCluskey method to determine simplified representations for the following Boolean functions. For three-variable functions, use the procedure of Examples 8 *and* 9; *for four-variable functions use Example* 10*'s tabular method.*

7.6.29. The function of Exercise 21
7.6.30. The function of Exercise 22
7.6.31. The function of Exercise 23
7.6.32. The function of Exercise 24
7.6.33. The function of Exercise 25
7.6.34. The function of Exercise 26
7.6.35. The function of Exercise 27
7.6.36. The function of Exercise 28

Exercises 37–39: Blocks in K-Maps
The following problems explore blocks of cells in K-maps.

7.6.37. *Boolean Expressions for Blocks in a Three-Variable K-map.*
a. Explain why in a K-map for three-variable functions a rectangular block of two adjacent 1-cells represents a product of two literals.
b. Explain why in a K-map for three-variable functions a rectangular block of four adjacent 1-cells represents a single literal.

7.6.38. *Boolean Expressions for Blocks in a Four-Variable K-map.*
a. Explain why in a K-map for four-variable functions a rectangular block of two adjacent 1-cells represents a product of three literals.

b. Explain why in a K-map for four-variable functions a rectangular block of four adjacent 1-cells represents a product of two literals.
c. Explain why in a K-map for four-variable functions a rectangular block of eight adjacent 1-cells represents a single literal.

7.6.39. *Boolean Expressions for Blocks in an N-Variable K-map.*
a. Generalize the results of Exercises 37 and 38: in a K-Map for an n-variable Boolean function, a rectangular block of 2^k adjacent 1-cells represents a product of _____ literals, for $k = 0, 1, \ldots, n-1$. What function does the full block of 2^n 1-cells represent?
b. Explain why your formula in part *a* is correct.

Exercises 40–42: Implicants for Boolean Functions
The following problems explore implicants for Boolean Functions.

7.6.40. *Implicants*
Claim: *if a function's implicant equals* 1*, then the function as a whole equals* 1.
a. Show from the definition for an implicant that xy is an implicant for $f(x, y, z) = x + z$. Then use this to illustrate the given claim.
b. Argue the given claim in general.
c. Explain why the term *implicant* is an appropriate one.
d. Is the converse of the above claim also true? What if the implicant is a prime implicant? an essential prime implicant?

7.6.41. *Maximal Blocks and Implicants*
a. Explain why maximal blocks in a K-map correspond to prime implicants. Must maximal blocks also be essential prime implicants?
b. Prove or disprove: the sum of a Boolean function's essential prime implicants is a Boolean expression that represents the function.

7.6.42. *The Quine-McCluskey Method and Prime Implicants*
Explain why all products that haven't been checked off in the tabular approach to the Quine-McCluskey method are prime implicants of the function.